FracNet:
Flow and Transport through Fracture Networks

Ruben Juanes[1] Christopher W. MacMinn[2]

[1]MIT

[2]University of Oxford

October 18, 2025

1 Introduction

This project concerns itself with fluid flow and solute transport in the subsurface, and specifically through fractured rocks. This is a very important topic for understanding problems like groundwater contamination due to chemical spills or surface runoff, the long-term storage of nuclear waste or carbon dioxide in the subsurface, and the development of geothermal resources.

1.1 The subsurface

The *subsurface* is everything below Earth's surface. In the context of engineering, we typically limit ourselves to Earth's crust, which is the outermost layer of rock and everything on it. The thickness of the crust ranges from about 20 to about 120 km, or from about 0.3% to about 2% of the radius of Earth. Very near to the surface (the top few metres or tens of metres), the crust consists of a mixture of soil, rock, and organic matter. Below this, the crust is solid rock.

Much of the shallow subsurface (the top few hundreds to thousands of meters of the crust) was formed through sedimentary processes, where small mineral particles and organic matter sink to the bottoms of rivers, lakes, and oceans and are then buried and compacted over geologic time. As a result, soil and most rocks are layered and porous. We refer to the open space between the solid grains as the *pore space*. The fraction of the volume that is taken up by pore space, or the *porosity*, is typically a few tens of percent.

1.2 Groundwater flows

In most of the subsurface, the pore space is completely filled with groundwater. This groundwater flows naturally through the subsurface due to a combination of tectonic activity and gravity. Groundwater generally flows downhill, and typically travels from the surface into

the subsurface, from the shallow subsurface into the deeper subsurface, and eventually from the subsurface into the oceans. These flows are central to the global water cycle.

1.3 Fractured rocks

Most natural rocks are heavily fractured due to tectonic stresses. Fractures are planar features consisting of two rough rock faces that are separated by a typical distance, or *aperture*, b.

1.4 Flow in fractured rocks

It is typically much easier for fluid to flow through these fractures than through the tiny pores between the rock grains. As a result, fracture networks act as high-permeability pathways that can dominate the flow. By analogy, think of traveling between two random points A and B in Boston. You could walk directly through the city from A to B; this might minimize the distance travelled, but it will be quite slow. Instead, you could walk to the nearest T station; you will be restricted to traveling along the T network, which may lead to a longer journey in terms of *distance travelled*, but you will still probably reach your destination much more quickly. Walking through the city is analogous to flow through the porous matrix of a rock (direct but slow), whereas travel by tube is analogous to flow through the fracture network (possibly indirect, but fast).

1.5 Flow in a single fracture

Suppose that two points i and j are connected by a fracture of length l_{ij}. Let p_i and p_j be the pressures at i and at j, respectively, and let q_{ij} be the fluid flux from i to j. Note that a flux is a volume per unit time *per unit cross-sectional area*. The flux through a planar fracture can then be calculated according to Darcy's Law or by solving the Stokes Equations, and is given by

$$q_{ij} = -\frac{b_{ij}^2}{12\mu}\left(\frac{p_j - p_i}{l_{ij}}\right), \tag{1}$$

where b_{ij} is the aperture of the fracture (the average distance between the fracture walls) and μ is the dynamic viscosity of the fluid ($\mu = 1 \times 10^{-3}\,\mathrm{Pa\,s}$ for water).

The volume flow rate from i to j, Q_{ij}, is then given by

$$Q_{ij} = b_{ij}\, q_{ij} = -\frac{b_{ij}^3}{12\mu}\left(\frac{p_j - p_i}{l_{ij}}\right). \tag{2}$$

We are working in two dimensions, so Q_{ij} is a volume per unit time *per unit depth into the page*. The flow travels from i to j if $p_i > p_j$ ($q_{ij} > 0$), and from j to i if $p_j > p_i$ ($q_{ij} < 0$). Note that $b_{ij} = b_{ji}$ and $l_{ij} = l_{ji}$ since the properties of the fracture do not depend on the direction of flow, and that $Q_{ij} = -Q_{ji}$ by definition.

It proves useful to define the flowrate in terms of the fracture transmissibility T_{ij},

$$Q_{ij} = T_{ij}(p_i - p_j), \quad T_{ij} = \frac{b_{ij}^3}{12\mu l_{ij}}, \tag{3}$$

1.6 This project

Your goal is to create a computational tool for simulating flow and transport through a fracture network. Although the real world is obviously three dimensional, you can work in two dimensions for simplicity. In addition, you can assume that fluid flows *only* through the fractures, and not through the porous matrix of the surrounding rock. This is a good assumption for many rocks, particularly for crystalline rocks like granite because they are brittle and not very porous (porosity of a few percent at most).

In 2D, you can approximate each fracture as a linear feature (long and thin). Each fracture can intersect with a number of other fractures, and we will refer to the point where two fractures intersect as a *node*, and the fracture segment connecting two nodes as a *link*. The set of fractures and nodes then forms a network. Your task is to simulate fluid flow and transport through this fracture network. In other words, you will be developing what is known as a Discrete Fracture Network (DFN) model.

1.7 Your report

Your report should briefly explain how you implemented each bit of the project below, and then discuss the results. In addition, please be sure to include the various figures that are specifically requested within the project description below.

Note that Tasks 6 and 7 involve implementing and studying transport. These are ***optional***. If you finish the rest in a comfortable amount of time, you are encouraged to try these—they are fun and interesting.

2 Tasks

2.1 Generate two families of random fractures

In 2D, a fracture can be fully characterized by its orientation, its length, and its aperture. Depending on the rock, fracture orientation can be relatively regular or quite random. Here we will focus on a typical case in which there are two dominant orientations: horizontal and vertical. For each family, fractures are generated by assigning a random location, length and aperture to each fracture.

Task 1a: Write a function to produce a list of two families of random fractures (horizontal or `x` fractures; vertical or `y` fractures), characterized by their locations, lengths and apertures. Note that:

- You should work in a rectangular domain of width L_x and height L_y.
- You should generate N_x horizontal fractures and N_y vertical fractures.
- For each fracture, the value of each quantity should be randomly selected from a uniform distribution.
- Specify the location of each fracture via the coordinates x and y of its midpoint, keeping these within the domain ($0 \leq x \leq L_x$ and $0 \leq y \leq L_y$).
- The length l of each fracture should be between $l_{x,\max}$ for horizontal fractures and $l_{y,\max}$ for vertical fractures.
- From the midpoint location and the fracture length, determine the initial and endpoint locations of each fracture
- The aperture b of each fracture should be between b_{mim} and $b_{\max}$.

To produce a standardized and modular computational tool, please follow these design conventions:

- Your function should accept the following inputs: the length L_x and height L_y of the rectangular domain; the number of horizontal fractures N_x and vertical fractures N_y; the maximum fracture length of horizontal fractures $l_{x,\max}$ and vertical fractures $l_{y,\max}$; and the minimum and maximum fracture aperture, $b_{\min}$ and $b_{\max}$.
- Your function should return the following vectors as outputs: The midpoints, initial and endpoints of horizontal fractures; the lengths of horizontal and vertical fractures, and the apertures of horizontal and vertical fractures. These should all be vectors of size N_x for horizontal fractures and N_y for vertical fractures.
- *Hint:* You may want to generate the midpoint coordinates such that the y-coordinates of horizontal fractures and the x-coordinates of vertical fractures are in ascending order.
- *Hint:* I used the following header for this function:

```
1 function [xfmid,xfini,xfend,yfmid,yfini,yfend,lfx,lfy,bfx,bfy] = ...
2     generate_fractures(Lx,Ly,Nx,Ny,lx,ly,bmin,bmax)
```

You can use the following function to plot the fractures and test your function:

```
function plot_fractures(figno,Lx,Ly,xfini,xfend,yfini,yfend)
%
% plot fractures

figure(figno)
plot([0 Lx Lx 0 0],[0 0 Ly Ly 0],'k-')
hold on
plot([xfini(:,1),xfend(:,1)]',[xfini(:,2),xfend(:,2)]','m-','LineWidth',1)
plot([yfini(:,1),yfend(:,1)]',[yfini(:,2),yfend(:,2)]','c-','LineWidth',1)
%
axis equal
axis tight
axis([0 Lx 0 Ly])

return
```

Also, you can use the following function to save the figure (feel free to choose a different file format for the output):

```
function save_figure(figno,figname)

% save figure
figure(figno)
set(gcf,'PaperPosition',[1 1 6 3])
print(gcf,'-dpng','-r600',[figname,'.png'])
% print(gcf,'-depsc','-loose',[figname,'.eps'])
```

Include a figure like this in your report for $L_x = 200\,\mathrm{m}$, $L_y = 100\,\mathrm{m}$, $N_x = 100$, $N_y = 200$, $l_{x,\max} = 60\,\mathrm{m}$, $l_{y,\max} = 30\,\mathrm{m}$. A typical realization of the two sets of fractures is illustrated in Figure 1.

Task 1b: Control the random number generator.

Modify your function `generate_fractures` to accept an additional input `seed`. This should be the last input in the list. You should also modify your function to return the value of `seed` as an additional output, again as the last one in the list. Your function should check whether `seed` is `NaN` and then do one of two things:

1. If `~isnan(seed)`, your function should use `seed` to seed MATLAB's random number generator. Hint: Type `help rng` at the MATLAB prompt.
2. If `isnan(seed)`, your function should `shuffle` MATLAB's random number generator and return the corresponding value of the seed as `seed`.

These modifications will allow you to save and reproduce a certain fracture network if

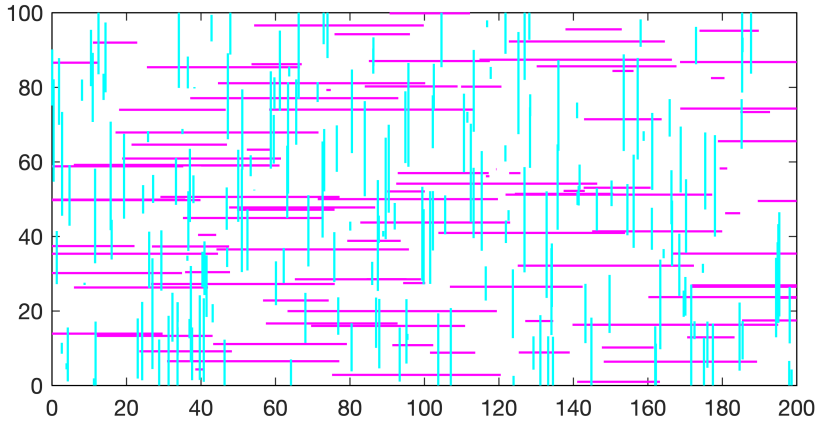


Figure 1. Realization of two sets of random fractures, horizontal (magenta) and vertical (cyan). It was obtained with the following values of the parameters: $L_x = 200\,\text{m}$, $L_y = 100\,\text{m}$, $N_x = 100$, $N_y = 200$, $l_{x,\max} = 60\,\text{m}$, $l_{y,\max} = 30\,\text{m}$.

you want to. To test, try running your code several times with `seed=NaN`; the network should look different every time. Then, set `seed` to an integer value of your choice and try running your code several times with that same seed value; you should get the same result every time.

2.2 Find the fracture intersections (nodes) and the fracture segments connecting them (links)

Typically, your two sets of fractures will have many intersections. In order to simulate flow through a network, you must now find these nodes.

Task 2a: Write a function to find the intersections between the two sets of fractures.

Given the locations of all the fractures in each set (horizontal and vertical), as specified by the vectors of midpoint, initial and endpoint coordinates, determine the list of all fracture–fracture intersections, as well as intersections of horizontal fractures with the $x = 0$ and $x = L_x$ boundaries *if* those fractures intersect with at least one vertical fracture.

For each node, store the vector of x- and y-coordinates, and the number of the intersecting x-fracture ($i_x \in \{1, \ldots, N_x\}$) and y-fracture ($i_y \in \{1, \ldots, N_y\}$). Determine the number N_{node} of nodes.

Hint: I used the following header for this function:

```
1 function [posnode,nfracx,nfracy,nnode] = ...
```

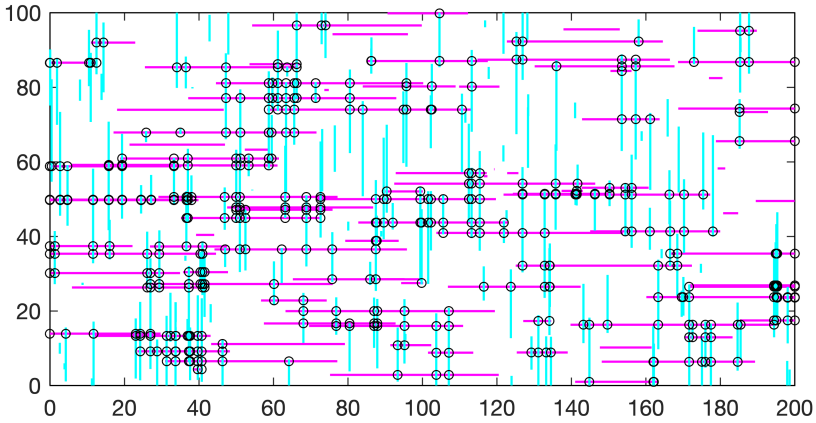


Figure 2. Same as Figure 1, but with nodes (fracture intersections) overlaid.

```
    fracture_intersections(Lx,Ly,Nx,Ny,xfmid,xfini,xfend,yfmid,yfini,yfend
        )
```

The nodes of the sets of fractures shown in Figure 1 are illustrated in Figure 2.

Task 2b: Write a function to determine links between nodes and their transmissibilities.

This function should take as inputs the outputs of function `fracture_intersections`, together with the `x` and `y` fracture apertures and the fluid viscosity μ. The output should be the list of links between nodes, that is, between immediate neighbor fracture intersections (and including intersections with the left and right boundaries). Each link should contain the following information: the origin and destination nodes, the link aperture, and the link transmissibility (Equation (3)).

Hint: Note that this provides a *directed* network.

Hint: I used the following header for this function:

```
function [linknods,linkapert,linktrans,nlink] = ...
    build_network(nnode,posnode,nfracx,nfracy,Nx,Ny,bfx,bfy,visc)
```

You should plot the links of the network. For this purpose, you can use the following function:

```
function plot_links(figno,nlink,linknods,posnode)
%
% plot links of fracture network

figure(figno)
for ilink = 1:nlink
    plot(posnode(linknods(ilink,:),1),posnode(linknods(ilink,:),2),...
```

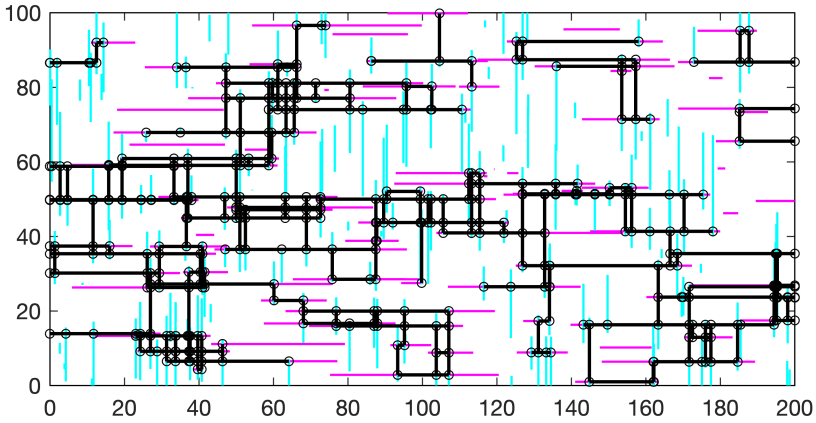


Figure 3. Same as Figure 2, but with links (fracture segments connecting nodes) overlaid.

```
        'k-','LineWidth',1.5)
end

return
```

In Figure 3 we plot an example of the links between nodes.

2.3 Determine the backbone of the fracture network

You might have noticed that the links connecting nodes do not always connect those nodes with the inlet ($x = 0$) and outlet ($x = L_x$) boundaries. From the point of view of flow, those node clusters are isolated. In this task, you will need to build the connected network (or backbone), which excludes those disconnected nodes.

Task 3: Write a function to find the (transmissibility-weighted) adjacency matrix of the network, and the list of nodes that belong to the backbone.

- The function should take as inputs the structure with the origin and destination nodes of all links (`linknods`), the list of link transmissibilities (`linktrans`), and the list of node coordinates (`posnode`).
- It should first build the transmissibility-weighted adjacency matrix for the entire network. This is a `sparse` matrix of dimensions $N_{\text{node}} \times N_{\text{node}}$, and it should be symmetric.
- It should also build a `graph` of the network.
- It should then determine the nodes in the network that are connected to the inlet nodes. You can first find the inlet nodes based on their x-coordinate ($x = 0$). Then, for all nodes in the network, you can compute the *minimum distance* to any of the inlet nodes (function `distances`): if that distance is `Inf`, it means that the node is

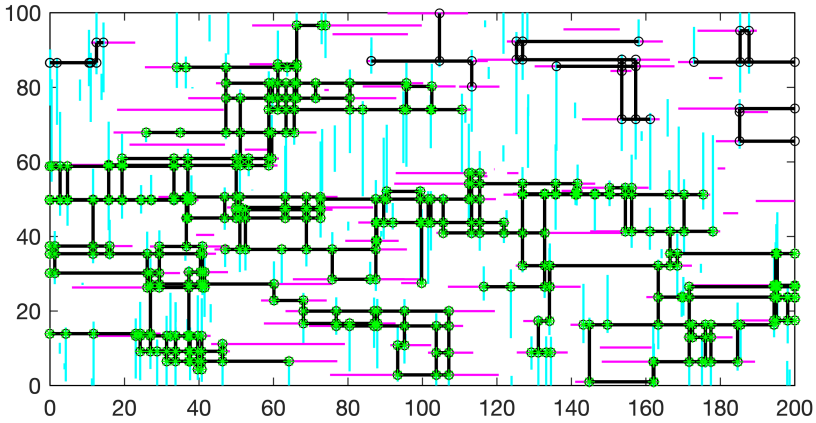


Figure 4. Same as Figure 3, but with backbone nodes (nodes connected to both inlet and outlet boundaries) overlaid with green symbols. Note the presence of isolated nodes, that is, nodes that are disconnected from either inlet *or* outlet boundaries.

not connected to the inlet.

- Repeat the procedure above for the outlet boundary (whose nodes satisfy that $x = L_x$).
- If the sum of the distances to inlet and to outlet is `< Inf`, then the node is in the backbone. Otherwise, it is isolated. The function should produce two vectors: a vector `backbonenods` of size N_{bbn} with the list of nodes in the network backbone; and a vector `isolatednods` of size $N_{\text{nnode}} - N_{\text{bbn}}$ with the isolated nodes (that is, the complement set of `backbonenods`).
- *Hint:* I used the following function header:

```
function [Adjmat,backbonenods,isolatednods,inletnods,outletnods] =
   ...
    backbone(linknods,linktrans,posnode,nnode,Lx)
```

In Figure 4 we indicate the nodes of the fracture network belonging to the backbone.

2.4 Derive and solve the flow equations

Now that we know everything about the network, we can use this to study fluid flow through the network. Suppose that the network is full of fluid, and we impose a pressure p_{in} at the left boundary ($x = 0$) and a lower pressure p_{out} at the right boundary ($x = L_x$). What is the resulting flow rate through the network? What is the flow rate through each individual fracture? What is the fastest path from left to right? What is the slowest path from left to right? To answer these questions, we have to derive and solve the equations for fluid flow on this network. These are based on conservation of mass and Darcy's Law.

Recall that Q_{ij} is the volume flow rate from node i to node j, such that $Q_{ij} > 0$ if the flow is from i to j and $Q_{ij} < 0$ if the flow is from j to i. If the fluid is incompressible and the

rock is rigid, conservation of mass requires that the net flow into each node must be zero. Mathematically, this can be written

$$\sum_j Q_{ij} = 0 \quad \text{for all } i. \tag{4}$$

This just means that the sum of the flows to/from each node must be zero, so that no node accumulates or loses fluid. This is exactly analogous to Kirchhoff's current law for circuits.

Recall from above that Darcy's Law (Eq. 2 or Eq. 3) provides a relationship between the volume flow rate Q_{ij} from i to j and the pressures at i and at j (p_i and p_j, respectively). Combining Darcy's Law (Eq. 3) with conservation of mass (Eq. 4), we arrive at

$$\sum_j T_{ij}(p_i - p_j) = 0 \quad \text{for all } i. \tag{5}$$

This is a linear system of N_{bbn} equations (one for each value of i), where the unknowns are the pressures p_i of nodes in the network backbone. However, we have fewer than N_{bbn} unknowns because the pressures at the nodes on the left and right boundaries are known (we don't know *a priori* how many of these there will be, as the network is random).

You may recall that any linear system of equations can be written as a single matrix equation of the form $\mathbf{Tp} = \mathbf{c}$, where $\mathbf{p}$ is the vectors of unknowns, $\mathbf{T}$ is the matrix of coefficients, and $\mathbf{c}$ is the right-hand side vector. The solution to this, in MATLAB, is simply `pvec = Tmat\cvec`. In order to get a meaningful solution, we must impose boundary conditions. This is done in several steps:

1. Generate the transmissibility matrix $\mathbf{T} = T_{ij}$, which contains the transmissibility of the links between nodes *in the network backbone*. Use the transmissibility-weighted adjacency matrix, but restrict it to the backbone nodes. The off-diagonal entries of this matrix are $-T_{ij}$. Then build the diagonal of this matrix such that mass conservation is satisfied at each node $i = 1, \ldots, N_{\text{bbn}}$.
2. Identify and store the inlet and outlet nodes that belong to the backbone. It is for these nodes that the boundary conditions must be imposed.
3. For all backbone inlet nodes (say, backbone node number i):
   (a) Find the backbone node j it is connected to.
   (b) Subtract the term $-T_{ji}p_{\text{in}}$ from the component c_j of the RHS vector.
   (c) Set component i of the RHS vector to be: $c_i = T_{ii}p_{\text{in}}$
   (d) Set to zero the (i, j) and (j, i) off-diagonal entries of the transmissibility matrix: $T_{ij} = T_{ji} = 0$.
4. Repeat this procedure for the backbone outlet nodes.

Now you can solve the system of equations to get the pressures at every backbone node (including those at the boundaries).

Task 4a: Write a function to solve the flow equations for a generic fracture network.

I used the following function header:

```
function [conninletnods,connoutletnods,pvec] = ...
    solve_flow(Adjmat,backbonenods,isolatednods,inletnods,outletnods,
       pinlet,poutlet)
```

The length of vector `pvec` should be the same as that of `backbonenods`. Since this function must identify the boundary nodes that are connected, it proves convenient to output those as well, in preparation for the next task.

You can use the following parameters: minimum and maximum fracture aperture $b_{\min} = 10^{-3}\,\mathrm{m}$, $b_{\max} = 5\times10^{-3}\,\mathrm{m}$; fluid viscosity $\mu = 10^{-3}\,\mathrm{Pa\,s}$; inlet boundary pressure $p_{\mathrm{in}} = 10^{5}\,\mathrm{Pa}$ and outlet boundary pressure $p_{\mathrm{out}} = 0\,\mathrm{Pa}$.

The last step in the flow solution is to determine the flowrate Q_{ij} and fluid velocity q_{ij} along each link ij. To accomplish this, one must perform a loop over all the links in the network, and for each link identify the origin node i and destination node j; if both nodes belong to the backbone, then Eq. (2) and Eq. (1) yield the flowrate and fluid velocity along that link, respectively.

Task 4b: Write a function to compute the flow rate and fluid velocity along each link of the network.

This function should also compute the total flow rate into the network, and check that it is equal to the flow out of the network, $Q_{\mathrm{in}} = Q_{\mathrm{out}}$.

I used the following function header:

```
function [linkflow,linkvel,conninletlinks,flow_in] = ...
    flow_velocity(linktrans,linkwidth,linknods,backbonenods,isolatednods,
       ...
    conninletnods,connoutletnods,pvec,tol)
```

The last input variable is simply a small parameter that can be used to check that $|Q_{\mathrm{in}} - Q_{\mathrm{out}}| <$ `tol` (e.g., `tol = 1e-12`).

Flow through the network shown in Figure 3 is illustrated in Figure 5. It was generated with the following function:

```
function plot_flow(figno,Lx,Ly,linkflow,linknods,backbonenods,posnode,pvec,
   tol)
%
% Plot pressure at nodes and flowrate along links

figure(figno)
plot([0 Lx Lx 0 0],[0 0 Ly Ly 0],'k-')
hold on
```

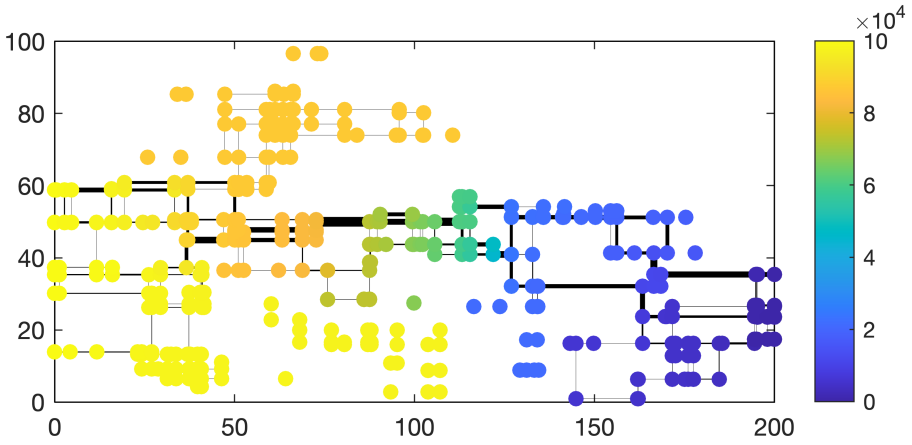


Figure 5. Flow through the network shown in Figure 3 for the following parameters: minimum and maximum fracture aperture $b_{\min} = 10^{-3}\,\mathrm{m}$, $b_{\max} = 5 \times 10^{-3}\,\mathrm{m}$; fluid viscosity $\mu = 10^{-3}\,\mathrm{Pa\,s}$; inlet boundary pressure $p_{\mathrm{in}} = 10^{5}\,\mathrm{Pa}$ and outlet boundary pressure $p_{\mathrm{out}} = 0\,\mathrm{Pa}$. Nodes are colored according to pressure (yellow to blue) and the thickness of links is proportional to flow rate.

```
8  axis equal
9  axis tight
10 axis([0 Lx 0 Ly])
11 %
12 maxlinewidth = 4;
13 linkwidth = abs(linkflow)/max(linkflow)*maxlinewidth;
14 for ilink = 1:length(linknods)
15     if abs(linkflow(ilink)) > tol
16         plot(posnode(linknods(ilink,:),1),posnode(linknods(ilink,:),2),...
17             'k-','LineWidth',linkwidth(ilink))
18     end
19 end
20 %
21 scatter(posnode(backbonenods,1),posnode(backbonenods,2),40,pvec,'filled')
22 colorbar
23
24 return
```

2.5 Study flow through the network

Now that you can generate a network and solve the corresponding flow equation, you can characterize the hydraulic properties of the network.

It is useful to set all the parameters defining the network at the very beginning of the script, e.g.:

```
% Flow and Transport through a Fracture Network
clear all
close all

%% Parameters
%
% domain dimensions
Lx = 200; % [m]
Ly = 100; % [m]
%
% number of fractures for each family
Nx = 100; % [m]
Ny = 200; % [m]
%
% maximum fracture length
lx = 60; % [m]
ly = 30; % [m]
%
% min and max fracture gap width
bmin = 1e-3; % [m]
bmax = 5e-3; % [m]
%
% fluid viscosity
visc = 1e-3; % [Pa.s]
%
% boundary pressures
pinlet = 1e5; % [Pa]
poutlet = 0; % [Pa]
%
% number of particles
Np = 1000;
%
% small tolerance
tol = 1e-12;
%
% plot or not / save figure or not
ioplot = 1;
iosave = 0;
```

Task 5a: Study a single network.

- Calculate the total flow rate through the network from left to right (the net rate of flow leaving the left boundary and arriving at the right boundary).

- Calculate the effective permeability of the entire network, $\bar{k} \equiv \bar{b}^2/12$, where $\bar{b}$ is the effective aperture of the entire network.
- Investigate the distributions of pressures and of flow rates within the network. What are the maximum, minimum, and mean values? What is the standard deviation? What do the distributions look like?

Task 5b: Study an ensemble of networks. In Task 5a, you generated a network, solved the flow equation, and calculated the total flow rate through the network. Now, write a script to repeat this calculation many times and collect the results. Use the same parameters, but make sure to use a different random network each time!) What are the maximum, minimum, and mean values of the total flow rate? What is the standard deviation? What does the distribution look like?

2.6 Implement particle transport on the network

To study solute transport, we need to simulate how flow carries solute through the network. Recall that flow travels through the network from left to right as long as $p_{\text{in}} > p_{\text{out}}$. As a result, any solute "particle" that is introduced at the left side (the inlet) will travel through the network and exit at the right side (the outlet). We are interested in the path that each solute particle takes, and the amount of time it takes for it to travel this path. By introducing many particles, we can characterize solute transport with statistics like the distribution of travel times. One way to simulate transport for a single particle is:

1. Randomly select an inlet node as the starting point. The probability of selecting a node should be proportional to the flow rate entering the domain through that node.
2. Move the particle to the next node. To do this,
   (a) Make a list of the candidate nodes. The candidate nodes are all nodes that are directly connected to the current node, and for which the flow is travelling from the current node toward the candidate node.
   (b) Select the next node from the list of candidate nodes. The selection should be random, but weighted by the relative flow rate toward each candidate node. (If 90% of the flow from node `3` is toward node `8` and 10% is toward node `5`, then a particle starting at node `3` is much more likely to move toward node `8` than toward node `5`.)
   (c) Move the particle, recording the identity of the next node and the time of arrival. The travel time can of course be calculated from the velocity and the distance, both of which are known.
3. Repeat step 2 until the particle reaches an outlet node.

Task 6a: Write a function to calculate the path of a single particle.

This function should take as inputs the flowrate and velocity of each link in the backbone, as well as the list of links between nodes and the node positions. It is also useful to pass as inputs quantities related to the input and output boundaries that have already been computed: the lists of nodes of the backbone on each boundary, and the corresponding list of links at the inlet boundary.

The outputs are two one-dimensional arrays of the same size: `npath` containing the node numbers of the travelled particle trajectory; and `tpath` with the corresponding time at which the particle passes through each node (the last element of `tpath` is the "breakthrough time" of that particle).

I used the following function header:

```
function [npath,tpath] = ...
    transport_one(linkflow,linkvel,linknods,posnode,...
    conninletnods,conninletlinks,connoutletnods,tol)
```

Task 6b: Write a function to calculate the paths of many particles.

This function performs a loop over a specified number of particles, N_p, and calls function `transport_one` each time. The function should output the paths of nodes `npaths` and corresponding transit times `tpaths` for all N_p particles. These should be cell arrays of size `Np` $\times$ 1 since the different particles do not necessarily take the same number of steps.

I used the following function header:

```
function [npaths,tpaths] = ...
    simul_transport(linkflow,linkvel,linknods,posnode,...
    conninletnods,conninletlinks,connoutletnods,tol,Np)
```

To test the function, plot the paths of several particles. Include a figure like this in your report. An example is shown in Figure 6. It was generated with the following function, which uses transparency so that the color of each path segment is more intense if it is part of the trajectory of multiple particles:

```
function plot_paths(figno,Lx,Ly,npaths,posnode)
%
% Plot particle paths
figure(figno)
plot([0 Lx Lx 0 0],[0 0 Ly Ly 0],'k-')
hold on
axis equal
axis tight
axis([0 Lx 0 Ly])
%
Np = length(npaths);
```

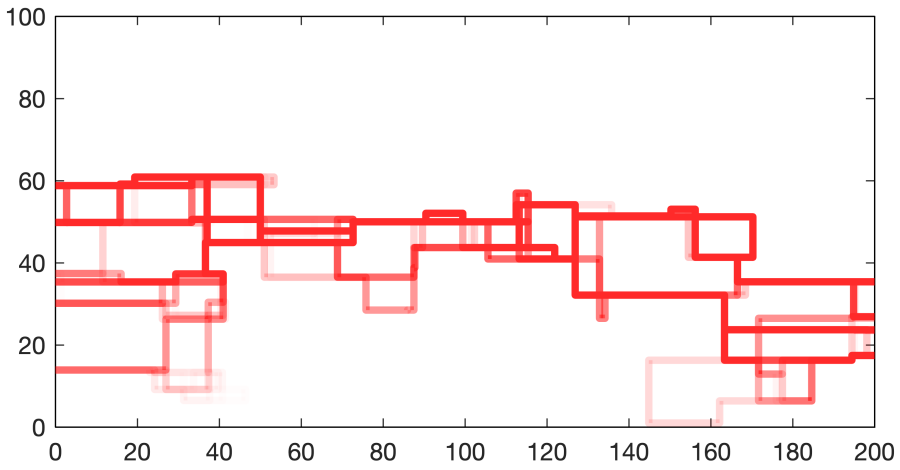


Figure 6. Particle paths (trajectories) of $N_p = 1000$ particles through the network shown in Figure 3 and corresponding to the flow solution shown in Figure 5. The color intensity of the links reflects the number of times that they are part of a particle path.

```
alpha_min = 0.01;
alpha_value = alpha_min + (1-alpha_min)*1/Np;
maxlinewidth = 3;
for ip = 1:Np
    plot(posnode(npaths{ip},1),posnode(npaths{ip},2),...
            'color',[1 0 0 alpha_value],'LineWidth',maxlinewidth)
end

return
```

2.7 Study the statistics of particle transport

Now that you can simulate transport, you can address many interesting questions. For example, you can study two classic "signatures" of transport:

1. The collective migration and dispersion of a plume of particles, represented in the characteristic space x vs t. This is illustrated in Figure 7 for the fracture network in Figure 5 and the $N_p = 1000$ trajectories of Figure 6. Because the transit times can vary wildly, the t axis is in log scale.

2. The distribution of particle breakthrough times (i.e., arrival times at the outlet boundary). In homogeneous porous media one expects a Gaussian distribution that reflects advection (proportional to the Darcy velocity) and diffusion (through "Fickian" dispersion). In heterogeneous media, like fracture networks, particle dispersion is far from Fickian (it's called "anomalous") and the breakthrough time distribution is far from Gaussian. It is instead much broader—often so broad that it needs to be represented in

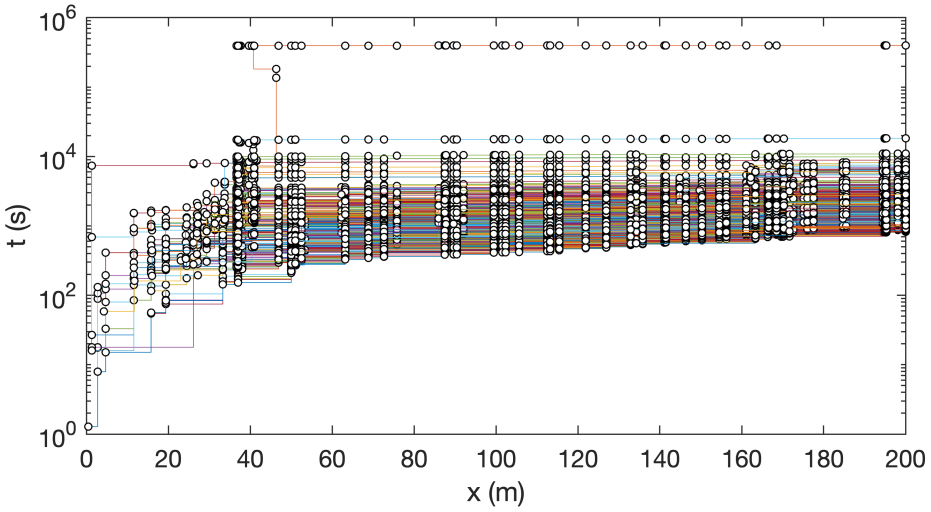


Figure 7. Space–time (x–t) plot of particle paths of $N_p = 1000$ particles through the fracture network in Figure 5. The color intensity of the links reflects the number of times that they are part of a particle path.

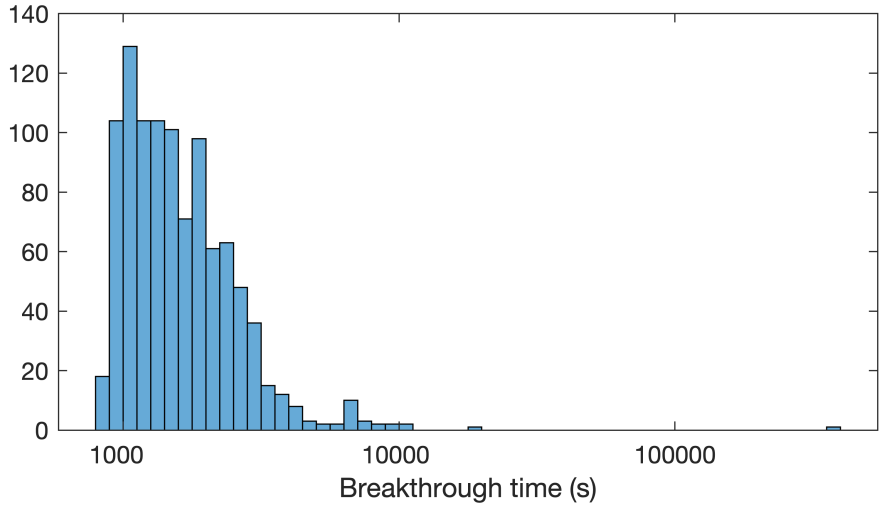


Figure 8. Breakthrough time distribution: histogram of particle arrival times at the outlet boundary for $N_p = 1000$ particles migration through the fracture network in Figure 5.

logarithmic scale. An example, for the same $N_p = 1000$ particle trajectories, is shown in Figure 8.

Task 7: You are asked to study in depth the transport signatures above, and address them in your report by combining quantitative results with discussion and interpretation.

Submission guidelines. Upload your the `.m` files as a zip file. This zip file should contain a main file `fracnet.m`, and all the functions necessary for the tasks above. Upload also a nicely typeset writeup with the figures from the simulations, and your analysis.